

SintonIA

Puesta en producción, CI/CD, IaC y operación

Proyecto SintonIA — MVP de chat documental con conocimiento propio
Carlota Moreno García

En este informe cierro el proyecto desde la perspectiva operativa pedida en la entrega final. No planteo una arquitectura cloud genérica, sino el estado real de SintonIA, qué parte puede considerarse preparada para una primera producción, qué queda todavía en preproducción y qué evidencias permiten verificarlo en Jira, GitHub y Vercel.

1. Resumen ejecutivo y alcance de producción

SintonIA es una aplicación web de chat documental construida con Next.js 16.2. El MVP permite autenticación OAuth vía Google, conversación con un backend propio integrado, chat conversacional basado en arquitectura RAG con recuperación documental mediante OpenAI File Search, citas visibles, persistencia de conversaciones y controles RGPD de exportación y borrado.

La decisión de alcance tomada ha sido conservadora ya que para producción incluiríamos el flujo de usuario final, pero no incluiríamos el panel administrativo como producción completa.

El primer despliegue real se centra en que una persona usuaria pueda entrar en la URL pública, autenticarse, formular preguntas sobre el corpus, recibir respuestas con respaldo documental cuando procede, conservar historial y ejercer sus derechos básicos de exportación o borrado.

El panel /admin/knowledge queda como preproducción operativa, este está implementado y protegido por roles expert/admin, pero todavía requiere conocer la URL manualmente y no incluye una experiencia completa de entrada visible, onboarding ni gestión autoservicio de administradores.

Fuera del alcance inmediato dejo, de forma explícita, la auditoría administrativa completa, el ensayo formal de restauración de Supabase, la operación multi-región y un modelo 24x7. No son requisitos proporcionados para este MVP, pero sí quedan identificados como pasos de madurez si el proyecto continuase.

2. Estado final del proyecto

El proyecto queda como un MVP desplegado y verificable. El núcleo de usuario final queda cerrado:

- Login
- Chat documental
- Streaming
- Citas
- Historial persistido
- RGPD
- Guardrails
- Límites de uso
- Dataset activo
- Despliegue público.

En Jira el trabajo restante se ha tratado como no bloqueante para el MVP evaluable, porque afecta principalmente a madurez operativa y administración interna, no al recorrido principal del usuario.

La versión de referencia actual es el commit a42a2d2 sobre main, con producción en Vercel en estado Ready. Se ha fijado el tag v0.1.0 para el primer pre-release bajo el título “**SintonIA v0.1.0: MVP público de chat documental**”, cerrando así la primera versión pública del MVP bajo un pre-release estable, manteniendo el commit en el informe para que la revisión sea auditable. Esta distinción es importante, el informe no inventa una madurez que no exista, sino que separa el estado funcional real de las mejoras operativas pendientes.

Las evidencias principales son el workflow quality en verde, el deployment de Vercel, los scripts test:release:config, test:release:vercel y test:release:rgpd con salida JSON ok:true, el README con comandos reales, el manifest documental, Jira y una demo final de cierre.

3. Arquitectura de despliegue y elección de servicios

La arquitectura elegida es gestionada y ajustada al alcance del MVP. Vercel ejecuta la aplicación Next.js y las API routes, Supabase separa Postgres transaccional y Storage privado, OpenAI aporta Responses API, Files, vector stores y File Search, Auth.js integra Google OAuth y la sesión. Esta elección evita operar máquinas virtuales, clústeres o una base vectorial propia, y encaja con el criterio de desplegar solo las piezas que puedo explicar, observar y revertir.

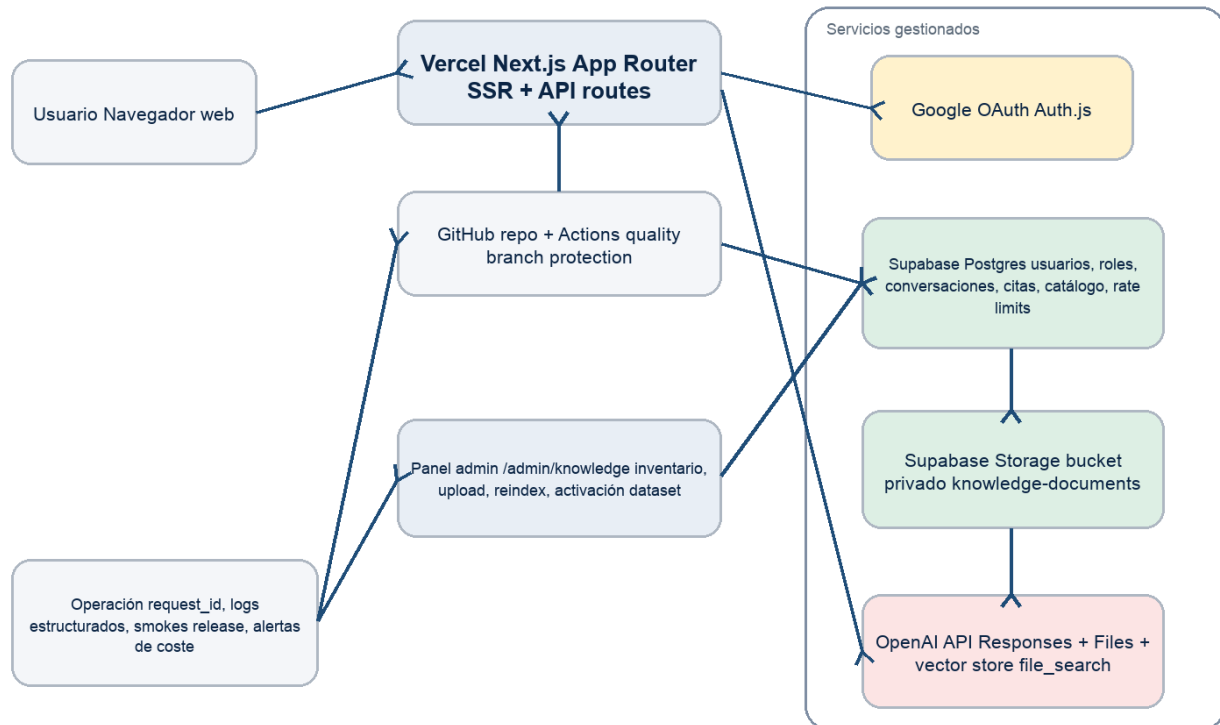
El navegador solo accede a la interfaz pública, al login y al chat. Las claves de Supabase y OpenAI permanecen en runtime server-side y no se exponen al cliente. Supabase Storage actúa como fuente canónica del corpus documental, mientras que OpenAI mantiene el índice operativo para retrieval. La dependencia de proveedores gestionados es aceptable para el MVP porque reduce carga operativa y coste inicial, si el uso creciera, habría que revisar límites, presupuestos, retención y lock-in del vector store.

Los secretos tales como API Keys han sido desplegados y almacenados de forma segura en el gestor de variables de entorno en Vercel. Se han separado variables de entorno de desarrollo con las de producción siguiendo las mejores prácticas recomendadas. Respecto a los costes operativos de la aplicación se han limitado únicamente al consumo de servicios de OpenAI vía API dejando un coste aproximado de dos centimos.

SintonIA: arquitectura de despliegue MVP

Producción controlada en Vercel + Supabase + OpenAI

Trazabilidad de cambio en Jira y GitHub; secretos solo en runtime server-side.



4. Entornos y estrategia de configuración

Actualmente distingo desarrollo local, Vercel Preview por rama y Vercel Production. Esta separación es suficiente para defender el MVP, pero no equivale a un modelo maduro de tres entornos. Antes de una producción más estable, añadiría una preproducción fija con alias propio, variables separadas y datos sintéticos.

La configuración de producción queda congelada en las tareas de release/configuración, incluyendo APP_BASE_URL, dataset activo, vector store, modelo, timeouts, chunking, prompt caching, límites de chat y rate limit. Los secretos reales no se documentan con valor ni se imprimen en evidencias, solo quedan referenciados como variables server-side. El fichero .env.example y el README sirven para reconstruir el entorno sin depender de memoria humana.

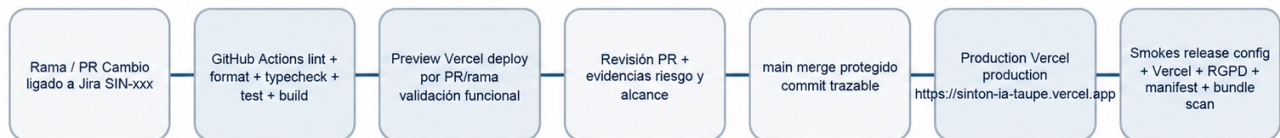
5. CI/CD y gestión de versiones

El flujo actual combina GitHub Actions para calidad y Vercel para despliegue. El workflow quality se ejecuta en push y pull_request, instala dependencias y valida lint, format:check, typecheck, tests y build. Vercel despliega automáticamente el commit validado, lo que permite conectar cambio de código, revisión y deployment sin pasos manuales opacos.

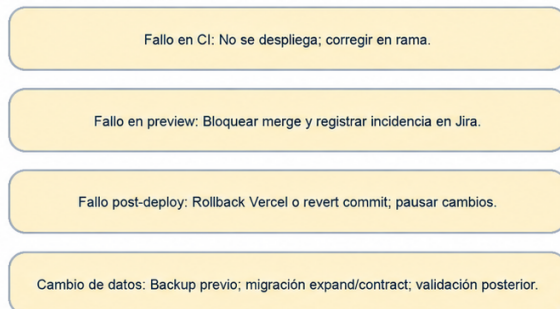
La promoción de versión la cierro con un criterio Go/No-Go sencillo: CI en verde, build correcto, scripts release con ok:true, URL pública accesible, login funcional, /chat anónimo redirigido, RGPD validado, ausencia de secretos en repo/bundle cliente y evidencia final accesible para el profesor. Si alguna condición bloqueante falla, no se publica la versión, si solo falta una mejora no crítica, se declara como riesgo residual.

Flujo CI/CD y promoción de versión

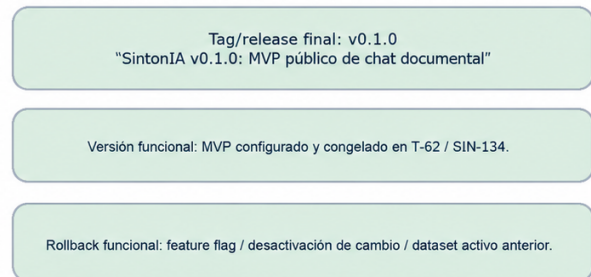
La versión solo avanza si genera evidencia automática y deja rollback claro



Guardrails de promoción



Versionado funcional



6. Infraestructura como código y reproducibilidad

SintonIA no tiene un stack Terraform. Para este MVP la reproducibilidad se apoya en artefactos más proporcionales, como migraciones Supabase versionadas, scripts de conocimiento, scripts de release, workflow de GitHub Actions, README, manifest documental y documentación de variables.

La siguiente mejora razonable sería representar con Terraform o Pulumi los recursos externos que hoy dependen de consola y documentación, como, alias de Vercel, variables no sensibles, budgets, alertas, log drains y registro de datasets. Los secretos no deberían almacenarse en laC, se referenciarían desde el proveedor correspondiente. Cualquier cambio que afecte datos, permisos o exposición pública debería pasar por plan revisable en PR y backup previo cuando aplique.

7. Puesta en producción, nuevas versiones y rollback

Para este alcance no considero necesario un blue/green completo ni multi-región. La estrategia adecuada es despliegue Git-based con Vercel, validación post-deploy y rollback al deployment anterior si una versión rompe el flujo principal. Vercel aporta deployments inmutables y permite volver rápido a una build anterior, Supabase requiere más cautela porque una migración de datos no se revierte igual que una build de frontend/backend.

Antes de promover una versión se valida CI, variables relevantes, login, redirecciones, chat, RGPD y configuración congelada. Tras el deploy se revisan home, /chat anónimo, /api/me, respuesta grounded, cabeceras no-store en RGPD y request_id. Para datos aplicaría estrategia expand/contract: añadir primero estructura compatible, desplegar código

compatible y retirar lo antiguo solo cuando ya no se use. Si una migración fuese destructiva, debería existir backup previo y ensayo de restauración en preproducción.

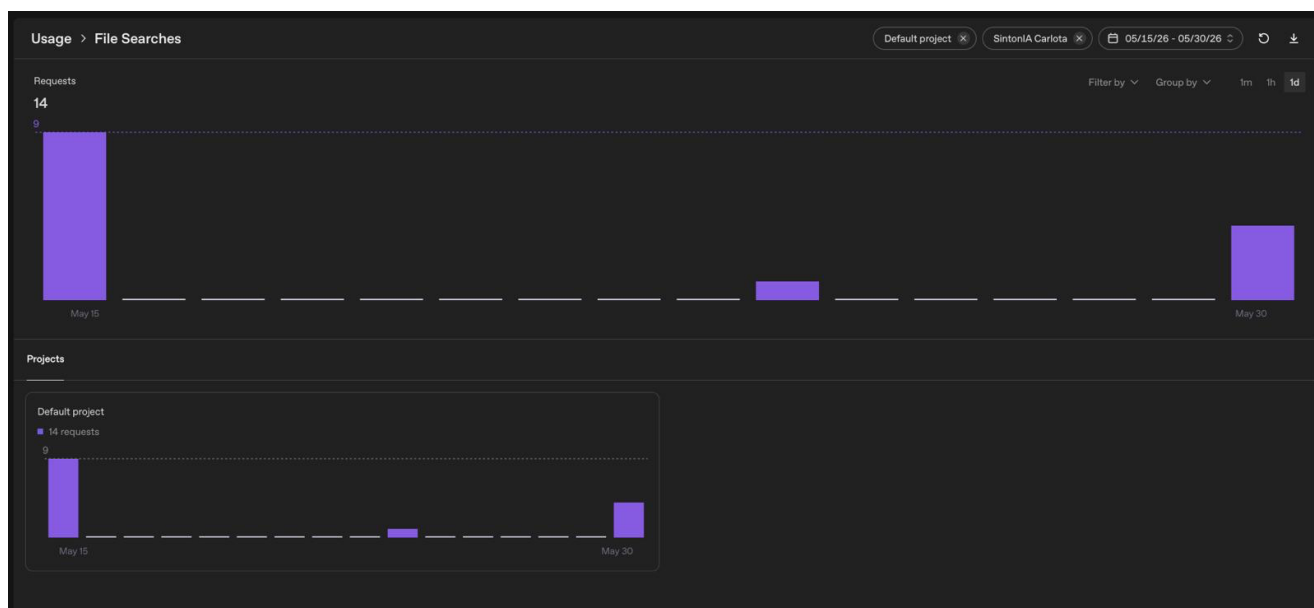
Como objetivos operativos para este MVP, acepto rollback de interfaz/API, recuperación de errores RGPD bloqueando release y reejecutando smokes, y contención de coste IA mediante rate limit, reducción de caps o pausa temporal del chat.

8. Observabilidad, SLOs y operación

La observabilidad disponible se basa en request_id, logs estructurados, incidentes de guardrails, smokes de release y dashboard de uso de OpenAI. Esta base permite reconstruir una petición o validar rutas críticas sin guardar secretos ni prompts crudos en evidencias. Lo que falta para una operación más madura es exportar logs runtime a un log drain/dashboard y definir post-mortems formales si se repiten incidentes.

Los SLOs del MVP son académicos, no contractuales: landing y login accesibles durante la ventana de evaluación, 100% de smokes críticos para chat y RGPD en cada release, respuestas grounded con cita esperada, errores 5xx por debajo de un umbral razonable en la ventana posterior al despliegue y coste mensual dentro del presupuesto. Para el control económico fijo un máximo de 20 €, aviso interno a 15 € y parada/revisión al llegar a 20 €, esto es suficiente para demo, smokes y margen de repetición.

La captura adjunta de OpenAI File Search muestra 14 requests entre el 15/05/2026 y el 30/05/2026 para el proyecto SintonIA, la cual interpreto como evidencia de uso real del retrieval.



En SintonIA, la gestión de incidentes se apoyaría en una observabilidad accionable: logs estructurados con request_id, métricas de errores y latencia, señales de negocio como disponibilidad del chat documental o del panel admin, y validaciones automáticas de release. Un incidente se abriría cuando hubiera degradación real del servicio, fallos en /api/chat, problemas de autenticación, errores RGPD, fallos de Supabase/OpenAI/Vercel o incidencias de guardrails. El flujo sería: detectar, clasificar la severidad, revisar logs y versión desplegada, mitigar con rollback, feature flag o dataset anterior, validar recuperación y dejar evidencia.

El postmortem no ha sido implementado, pero en el caso de implementarlo debería resumir qué ocurrió, cuándo se detectó, qué impacto tuvo, cuál fue la causa raíz, qué señales ayudaron, qué faltó y qué acciones correctivas se harán después. Así, la observabilidad no queda como un panel decorativo, sino como una herramienta para decidir cuándo actuar, cuándo pausar cambios y cómo mejorar la fiabilidad del MVP con trazabilidad.

9. Seguridad, privacidad y control de costes

La seguridad está integrada en el diseño del MVP: OAuth con Google, roles persistidos, rutas protegidas middleware, secretos server-only, rate limiting, guardrails, límites de entrada/salida y endpoints RGPD. El principio aplicado es mínimo privilegio: el usuario normal no accede al panel administrativo, las operaciones de corpus quedan restringidas a expert/admin y las claves de proveedor no se envían al navegador.

Los datos personales tratados son principalmente email, nombre/avatar de Google, rol, conversaciones, citas, logs técnicos y eventos de guardrails. La política operativa que aplicaría es minimizar exposición: no incluir secretos ni PII innecesaria en logs o capturas, conservar logs solo el tiempo necesario para diagnóstico (30 días), no persistir exportaciones RGPD y mantener trazabilidad de acciones administrativas sobre corpus cuando el panel pase a producción completa.

Para estabilidad operativa limito tokens, longitud de entrada, contexto enviado al modelo, timeouts y reintentos. Estos clamps evitan esperas indefinidas, costes inesperados y abuso del chat. El presupuesto de 20 € no sustituye los límites técnicos, pero da una frontera explícita de decisión: si se alcanza el aviso de 15 €, se revisa usage, si se alcanza 20 €, se pausa o reduce el servicio hasta entender el gasto.

SintonIA no implementa cifrado propio de mensajes o documentos a nivel de aplicación, delega el cifrado en tránsito y en reposo en Vercel, Supabase, OpenAI y Google, y refuerza la seguridad con bucket privado, variables server-only, sesiones protegidas y seudonimización de usuarios en logs.

Los riesgos principales son exponer secretos, usar permisos demasiado amplios, dejar rutas administrativas o documentos mal protegidos, registrar datos personales en logs, romper los flujos RGPD o disparar costes por exceso de llamadas a OpenAI, tokens o reindexaciones. Para reducirlos, el proyecto usa roles, rutas protegidas, rate limiting, límites de tokens, configuración de release congelada, smokes de producción y control de errores.

10. Evidencias y trazabilidad

- **Jira del proyecto:** <https://sintonia-proy4.atlassian.net>
- **Repositorio GitHub:** <https://github.com/carlotamoreeno/SintonIA>
- **Commit de referencia:** <https://github.com/carlotamoreeno/SintonIA/commit/a42a2d2257d0f3a8e57dd04adf6e921b71b7ec8f> . Estado validado antes del tag/release final
- **Tag/release:** <https://github.com/carlotamoreeno/SintonIA/tags>
- **Workflow quality:** <https://github.com/carlotamoreeno/SintonIA/actions/workflows/quality.yml> — validación automática de cambios
- **Run CI verde:** <https://github.com/carlotamoreeno/SintonIA/actions/runs/26372055190> — ejecución quality main success
- **Producción Vercel:** <https://sinton-ia-taupe.vercel.app>
- **Scripts release:** <https://github.com/carlotamoreeno/SintonIA/tree/main/tools/release> — test:release:config, test:release:vercel y test:release:rgpd
- **Migraciones Supabase:** <https://github.com/carlotamoreeno/SintonIA/tree/main/supabase/migrations> — schema reproducible y revisable
- **README:** <https://github.com/carlotamoreeno/SintonIA/blob/main/README.md> — instalación, ejecución, validación y operación documental
- **Tickets DT:** Se encuentran en Jira en el Backlog
- **Demo final de cierre:** <https://youtu.be/XVla02ttffs>

- Demo Sprint 1: <https://youtu.be/0wUVKn35wfo>
- Demo Sprint 2: <https://youtu.be/FCOHOhpoPXM>
- Demo Sprint 3: <https://youtu.be/SpOufMRwzW0>
- Evidencias Aplicación:

